Progress update

Making quantum circuits shorter, and proving how short they can get

Exact ground truth for local rewriting, and a general hardware-aware optimiser

Arnav Kapoor

IISER Bhopal

Building on B. Rosenhahn, T. J. Osborne, C. Hirche, *Optimization driven quantum circuit reduction*, arXiv:2502.14715 and New J. Phys. **27**, 104509 (2025)

Part I

The method we start from

Replace short windows of a circuit with shorter equivalents from a precomputed database.

Why shorten circuits at all?

On noisy hardware every gate can fail. A standard estimate of the chance that the whole circuit runs correctly is

$$\mathrm{ESP} = \prod_{\text{gates } g} \big(1 - e(g)\big).$$

Error rates per gate (medians)

	1q gate	2q gate	2q / 1q
IBM Heron*	$2.7 \cdot 10^{-4}$	$4.1 \cdot 10^{-3}$	**15**×
trapped ion†	$3 \cdot 10^{-5}$	10^{-3}	**33**×

*FakeTorino calibration, 133 qubits, 278 couplers. †model rates used in the paper.

Fewer gates, higher ESP

gates, each 99 %	ESP
100	$0.99^{100} = 37\,\%$
50	$0.99^{50} = 61\,\%$

Error rates: `experiments/make_talk_figs.py errors` (Qiskit FakeTorino calibration) and the ion model in the paper, Sec. 3. Figure: arXiv:2502.14715, Fig. 1

Why shorten circuits at all?

On noisy hardware every gate can fail. A standard estimate of the chance that the whole circuit runs correctly is

$$\mathrm{ESP} = \prod_{\text{gates } g} \big(1 - e(g)\big).$$

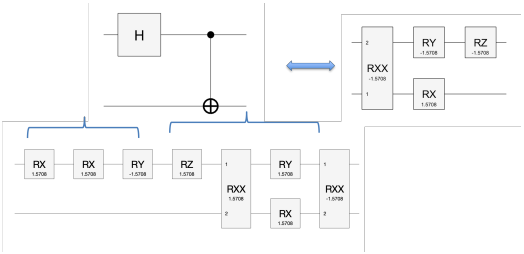
Error rates per gate (medians)

	1q gate	2q gate	2q / 1q
IBM Heron*	$2.7 \cdot 10^{-4}$	$4.1 \cdot 10^{-3}$	**15**×
trapped ion†	$3 \cdot 10^{-5}$	10^{-3}	**33**×

*FakeTorino calibration, 133 qubits, 278 couplers. †model rates used in the paper.

Fewer gates, higher ESP

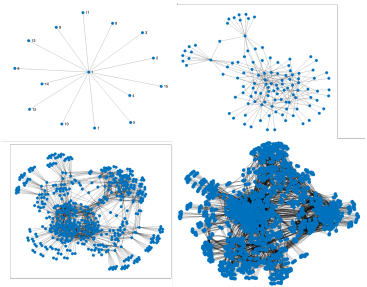
gates, each 99 %	ESP
100	$0.99^{100} = 37\,\%$
50	$0.99^{50} = 61\,\%$



H + CNOT compiled to ion-trap gates (8 gates) and the shortest equivalent (4 gates).

Error rates: `experiments/make_talk_figs.py errors` (Qiskit FakeTorino calibration) and the ion model in the paper, Sec. 3. Figure: arXiv:2502.14715, Fig. 1

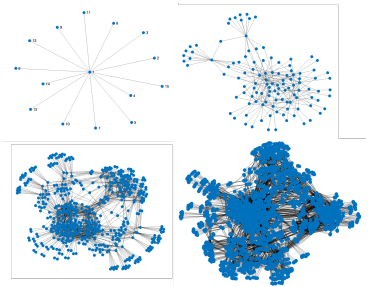
The method: a database and a sliding window



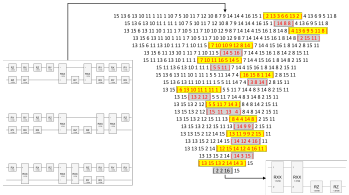
Compute graph. Nodes are circuits (as matrices), edges add one gate. Everything within radius r of the identity is stored with its shortest form. The paper uses $r = 5$–6.

Figures from arXiv:2502.14715, Figs. 2 and 3

The method: a database and a sliding window



Compute graph. Nodes are circuits (as matrices), edges add one gate. Everything within radius r of the identity is stored with its shortest form. The paper uses $r = 5$–6.



Window. Take 2–8 gates on ≤ 3 wires, look them up, substitute a shorter word, shuffle commuting gates, repeat.

Takeaway. Two knobs decide everything later: the window length w and the database radius r.

Figures from arXiv:2502.14715, Figs. 2 and 3

Part II

Exact ground truth

The gates behind the paper's benchmarks are Clifford, so the search space is a finite group we can enumerate completely.

Where the Clifford claim comes from: their Table I

Qubits	#Operators	Depth	Nodes	Edges
2	14	1	15	14
2	14	2	114	210
2	14	3	584	1596
2	14	4	2024	8176
2	14	5	4512	28336
2	14	6	7420	63168
3	24	1	25	24
3	24	2	337	600
3	24	3	3215	8088
3	24	4	23622	77160
3	24	5	137572	566928

TABLE I. Full Compute Graphs for different depths and amount of operators

arXiv:2502.14715, p. 4, Table I (cropped, unchanged)

Where the Clifford claim comes from: their Table I

Qubits	#Operators	Depth	Nodes	Edges
2	14	1	15	14
2	14	2	114	210
2	14	3	584	1596
2	14	4	2024	8176
2	14	5	4512	28336
2	14	6	7420	63168
3	24	1	25	24
3	24	2	337	600
3	24	3	3215	8088
3	24	4	23622	77160
3	24	5	137572	566928

TABLE I. Full Compute Graphs for different depths and amount of operators

arXiv:2502.14715, p. 4, Table I (cropped, unchanged)

Only $\pm\frac{\pi}{2}$ angles give these operator counts

	2 qubits	3 qubits
$R_X, R_Y, R_Z(\pm\frac{\pi}{2})$ on each qubit	$3 \cdot 2 \cdot 2 = 12$	$3 \cdot 2 \cdot 3 = 18$
$R_{XX}(\pm\frac{\pi}{2})$ on each pair	2	6
total	**14**	**24**

Our enumeration reproduces every row, node and edge counts identical, e.g. 137,572 nodes at depth 5. The authors' demo files also contain only $\pm\frac{\pi}{2}$ angles.

$\pm\frac{\pi}{2}$ rotations and $R_{XX}(\frac{\pi}{2})$ map Pauli operators to Pauli operators, which is the definition of a **Clifford** gate.

Check: `test/test_clifford.py::test_compute_graph_matches_table_one`, reproduction report Sec. 2

The journal version adds $\pm\frac{\pi}{4}$: handled as Clifford+T

a parameter θ. Four discretized angles $[-\frac{\pi}{2}, -\frac{\pi}{4}, \frac{\pi}{4}, \frac{\pi}{2}]$ are taken for the moment. For three qubits $[1, 2, 3]$ it is now possible to generate 12 gates, namely,

New J. Phys. 27, 104509 (2025), p. 7, Sec. 3.1 (cropped, unchanged)

$\pm\frac{\pi}{2}$ only (Table I, Tables III/IV)

Clifford group, finite. Enumerated completely, optimum known for every circuit.

with $\pm\frac{\pi}{4}$ (journal grid)

Clifford+T, infinite group. Exact distances by exact arithmetic up to a certified length, plus planted benchmarks.

How the code checks it

Every gate is tested before the search starts

```
# qcr/tableau.py, Tableau.row_table
for b in range(4**n):            # every Pauli P
    Q = U @ P[b] @ U^†           # conjugate it
    for c in range(4**n):
        ov = <P[c], Q> / 2**n
        if abs(abs(ov) - 1) < 1e-8: # Q = +-P[c]
            ...; break
    else:
        raise ValueError("gate is not Clifford")
```

Then the whole group is enumerated

```
# results/cayley3_ion_pm.log (last lines)
n=3 depth 15: +37,888 (total 92,891,648) 1489s
n=3 depth 16: +5,120  (total 92,896,768) 1491s
n=3 depth 17: +512    (total 92,897,280) 1492s
```

How the code checks it

Every gate is tested before the search starts

```
# qcr/tableau.py, Tableau.row_table
for b in range(4**n):              # every Pauli P
    Q = U @ P[b] @ U^†             # conjugate it
    for c in range(4**n):
        ov = <P[c], Q> / 2**n
        if abs(abs(ov) - 1) < 1e-8: # Q = +-P[c]
            ...; break
    else:
        raise ValueError("gate is not Clifford")
```

Then the whole group is enumerated

```
# results/cayley3_ion_pm.log (last lines)
n=3 depth 15: +37,888 (total 92,891,648) 1489s
n=3 depth 16: +5,120  (total 92,896,768) 1491s
n=3 depth 17: +512    (total 92,897,280) 1492s
```

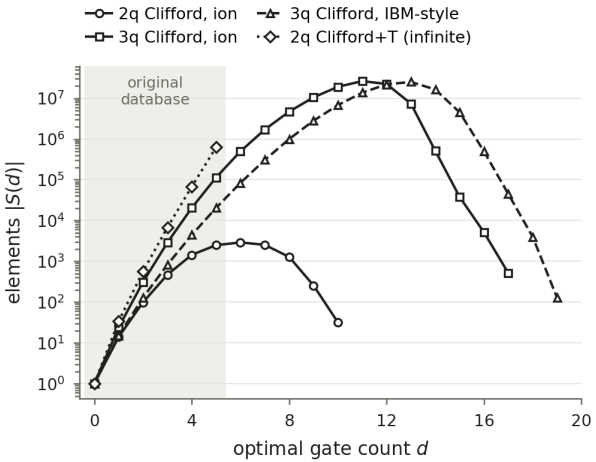
Group size: enumeration vs. formula

n	found	$2^{n^2+2n}\prod_j(4^j-1)$	test
1	24	24	**pass**
2	11,520	11,520	**pass**
3	92,897,280	92,897,280	**pass**

3 qubits	ion	IBM-style
diameter (longest optimum)	17	19
mean optimal length	10.72	12.34
enumeration time	25 min	27 min

`experiments/ext_cayley3.py`, `results/cayley3_{ion_pm,nisq}.json`, `test/test_clifford.py`

The whole group, sorted by optimal length



Elements by optimal length (3 qubits). The paper's depth-5 database (shaded) covers 0.15 % of the group.

- Every 3-qubit circuit over this gate set has a **proven shortest length**, looked up in one step.
- The paper's database stores 137,572 of 92,897,280 elements, $137{,}572/92{,}897{,}280 = 0.15\,\%$.
- The table is only a **referee**. The optimiser we test later never uses it.

How far from optimal is the paper's method?

Mean gate count, 3 qubits, 300 random gates, 20 circuits per row

	optimum	paper	ratio	Qiskit L3
ion trap	**10.4**	47.1	**4.5**×	127.2
IBM-style	**12.2**	36.8	**3.0**×	104.0

`results/ext_gap3_{ion,nisq}.json`, paper Table 3

How far from optimal is the paper's method?

Mean gate count, 3 qubits, 300 random gates, 20 circuits per row

	optimum	paper	ratio	Qiskit L3
ion trap	**10.4**	47.1	**4.5**×	127.2
IBM-style	**12.2**	36.8	**3.0**×	104.0

Is our port faithful?

ion trap, 3q	published	our port
Table III (paper)	45	47.1
Table I, all 11 rows		identical

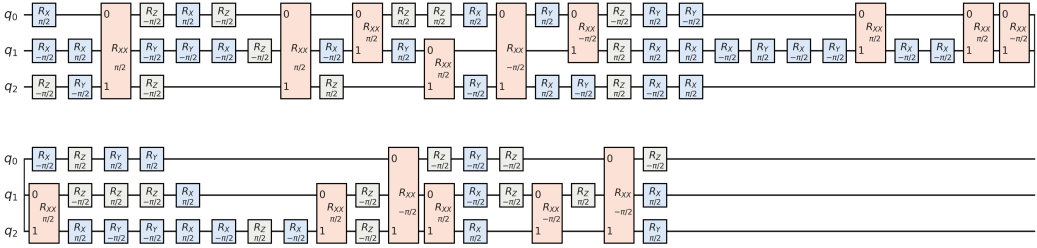
Reproduction report, Tables III/IV.

Takeaway. On its own benchmark the method returns 3–4.5 times the proven optimum. The port matches the published number, so the gap is the method, not our code.

`results/ext_gap3_{ion,nisq}.json`, paper Table 3

One circuit: the input

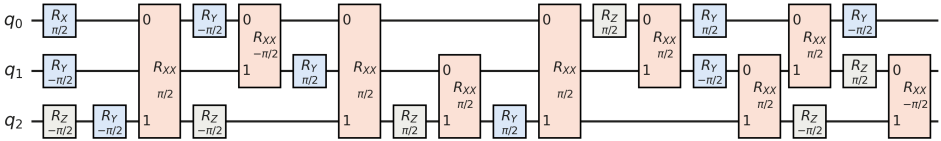
(a) Random 3-qubit word over the ion gate set, **80 gates**



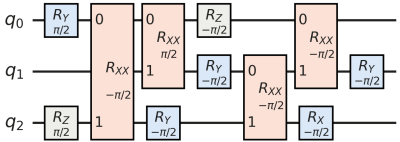
Gate lists in `results/talk_example.json`, drawn by `experiments/make_talk_figs.py example`

One circuit: the paper's method vs. the optimum

(b) Output of the paper's method, database radius 5: **24 gates**

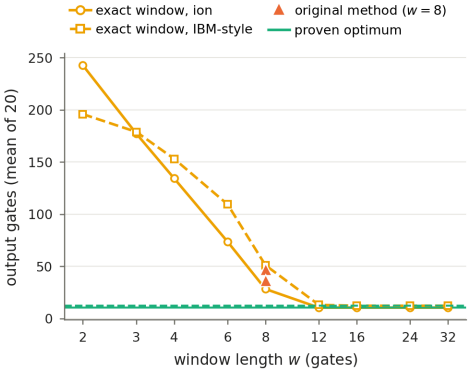


(c) Proven optimum, looked up in the exact table: **11 gates**



Both diagrams at the same scale. All three circuits are the same unitary up to phase.

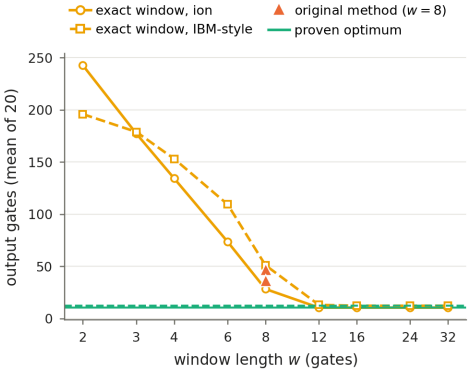
Why it falls short: the database is too small



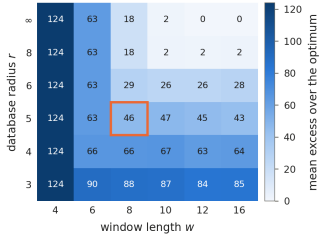
With exact lookups, a window of $w \geq 12$ (ion) or 16 (IBM-style) reaches the proven optimum on all 40 circuits.

`results/ext_grid_ion.json`, paper Fig. 3

Why it falls short: the database is too small



With exact lookups, a window of $w \geq 12$ (ion) or 16 (IBM-style) reaches the proven optimum on all 40 circuits.



Mean excess over the optimum by window w and radius r. The paper's $(8, 5)$ in red.

- Larger window at $r = 5$: excess 46 $\rightarrow$ 43.
- Larger database at $w = 8$: 46 $\rightarrow$ 18.
- $w \geq 12$ and $r \geq 8$: excess ≤ 2.

Takeaway. The database radius is the binding constraint. Our theorem predicts that both knobs must be large enough.

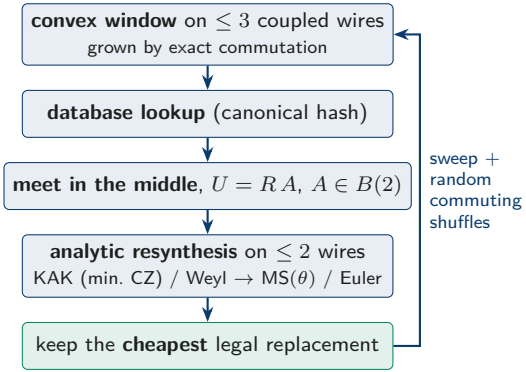
`results/ext_grid_ion.json`, paper Fig. 3

Part III

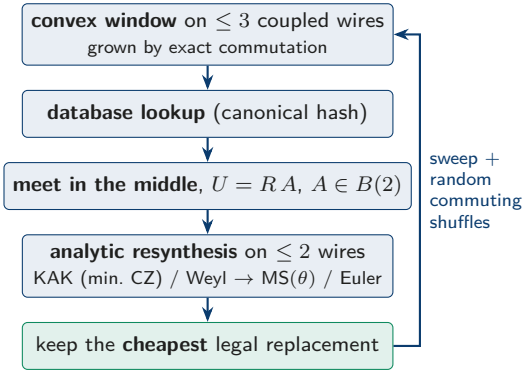
A general optimiser

Drop the Clifford crutch and add continuous angles, connectivity, calibrated noise, and exactness including phase.

The engine


convex window on ≤ 3 coupled wires
grown by exact commutation
database lookup (canonical hash)
meet in the middle, $U = R\,A$, $A \in B(2)$
analytic resynthesis on ≤ 2 wires
KAK (min. CZ) / Weyl $\rightarrow$ MS(θ) / Euler
keep the **cheapest** legal replacement
sweep + random commuting shuffles

The engine



- **Cost** $c_{\mathrm{cal}}(g) = -\log(1 - e(g))$ from calibration, so minimising cost maximises ESP.
- **Exact including global phase**: phases are tracked and repaired once at the end.
- **Never worse** than its input, by construction.
- **Any gate set**, through databases built per device (next slide).
- Runs *after* any compiler, as a post-pass.

The databases behind the engine

device	local topology	equivalence	depth	classes	build time
IBM Heron	line	up to phase	9	6.31 M	6 min
IBM Heron	line	output-aware	10	0.34 M	$<$1 min
IBM Heron	line	input-aware	14	1.41 M	$<$1 min
trapped ion	triangle	up to phase	6	28.4 M	38 min
trapped ion	triangle	output-aware	6	7.36 M	27 min
trapped ion	triangle	input-aware	7	31.6 M	23 min
paper ion ($\pm\frac{\pi}{2}$)	triangle	up to phase	9	2.93 M	7 min
paper IBM ($\pm\frac{\pi}{2}$)	triangle	up to phase	10	1.84 M	2 min

- A *class* is all 3-qubit circuits that are the same up to the stated equivalence, stored once by a 64-bit hash.
- Compare depth 9–10 here with depth 5 in the original paper. This is the knob from the previous part.

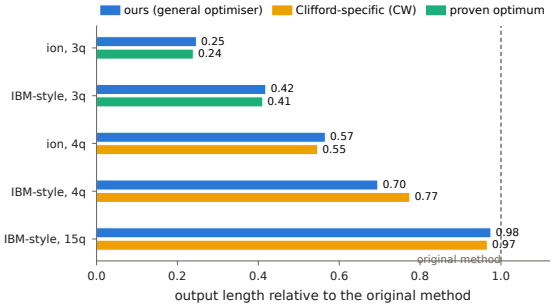
Paper Table B3, `experiments/gen_build_dbs.py`, `results/gdb_info.json`

Part IV

Results

The paper's benchmarks against ground truth, 60 algorithm circuits on hardware models, circuits up to 100 qubits, other tools, and real hardware.

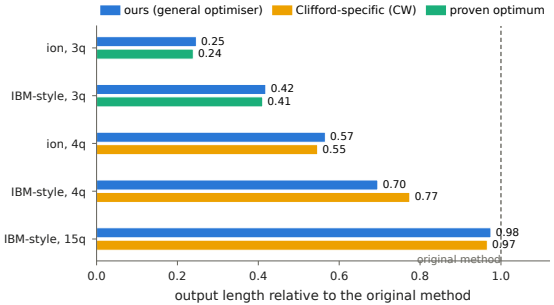
Result 1. The paper's benchmarks, without Clifford knowledge



Output length relative to the paper's method, same circuits and seeds.

Paper Table 3 and Fig. 4

Result 1. The paper's benchmarks, without Clifford knowledge



Output length relative to the paper's method, same circuits and seeds.

mean gates	paper	ours	optimum	#opt.
ion, 3q	47.1	**10.8**	10.4	17/20
IBM-style, 3q	36.8	**12.5**	12.2	18/20
ion, 4q	78.8	**46.1**	$\geq$11.1	
IBM-style, 4q	76.0	**51.4**	$\geq$12.9	
IBM-style, 15q	335.9	327.7		

Clifford+T, 2q	paper	ours
random, optimal on	81/99	**99/99**
planted, optimal on	40/41	**41/41**

4q: Clifford-specific optimiser 45.6 / 49.3, so ours is on par. 15q: Qiskit L3 332.1, all within 4 %.

Takeaway. Proven optimum on 35/40 three-qubit circuits without knowing the group. Large databases and convex windows are what matter.

Paper Table 3 and Fig. 4

Result 2. 60 algorithm circuits, relative to Qiskit level 3

	IBM Heron (58 circuits)			**trapped ion** (58 circuits)	
pipeline	2q gates	all gates	ESP	2q gates	all gates
TKET	×1.02	×1.71	×1.16	×1.27	×1.79
ours, exact, from Qiskit L1	×1.00	**×0.84**	**×1.21**	**×0.95**	**×0.76**
ours, exact, from Qiskit L3	**×0.98**	**×0.87**	×1.03	×1.00	×1.00
BQSKit L3 on L1 (approximate)	**×0.85**	×1.20	**×1.22**	**×0.94**	**×0.71**
ours, exact, on BQSKit L3	**×0.85**	×1.01	**×1.25**	**×0.94**	**×0.70**

Paper Table 4. Seed 0. Over 5 seeds the gate ratios move by at most 0.5 % (Result 4). Nam/VOQC, QASMBench, QFT, QAOA, Trotter, EfficientSU2, Steane, random, 2–12 qubits.

Result 2. 60 algorithm circuits, relative to Qiskit level 3

	IBM Heron (58 circuits)			**trapped ion** (58 circuits)	
pipeline	2q gates	all gates	ESP	2q gates	all gates
TKET	×1.02	×1.71	×1.16	×1.27	×1.79
ours, exact, from Qiskit L1	×1.00	**×0.84**	**×1.21**	**×0.95**	**×0.76**
ours, exact, from Qiskit L3	**×0.98**	**×0.87**	×1.03	×1.00	×1.00
BQSKit L3 on L1 (approximate)	**×0.85**	×1.20	**×1.22**	**×0.94**	**×0.71**
ours, exact, on BQSKit L3	**×0.85**	×1.01	**×1.25**	**×0.94**	**×0.70**

- **Stacked on L3** ours never adds a 2q gate and never lowers ESP. All gates $\times 0.87$ means 13 % removed.
- **BQSKit** finds better 2q structure but is approximate and leaves many 1q gates. Ours on top keeps its 2q savings and goes from $\times 1.20$ to $\times 1.01$, **16 %** fewer gates, exactly.

Paper Table 4. Seed 0. Over 5 seeds the gate ratios move by at most 0.5 % (Result 4). Nam/VOQC, QASMBench, QFT, QAOA, Trotter, EfficientSU2, Steane, random, 2–12 qubits.

Which objective? Let noisy simulation decide

pipeline / objective	mean TVD (49 circuits) Heron	ion
Qiskit L3	0.137	0.0091
ours, total count	0.096	0.0092
ours, 2q-weighted	**0.088**	**0.0084**
ours, calibrated	**0.088**	**0.0084**

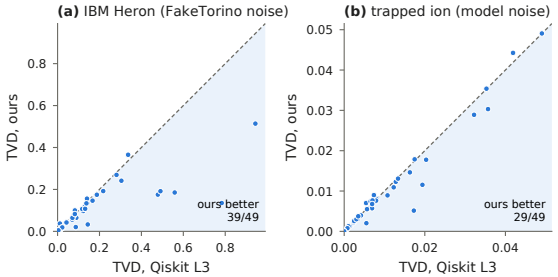
TVD is the distance between ideal and noisy output distributions (0 is perfect). Exact density-matrix simulation with FakeTorino errors (Heron) or a depolarising model (ion).

Paper Table B2

Which objective? Let noisy simulation decide

pipeline / objective	mean TVD (49 circuits) Heron	ion
Qiskit L3	0.137	0.0091
ours, total count	0.096	0.0092
ours, 2q-weighted	**0.088**	**0.0084**
ours, calibrated	**0.088**	**0.0084**

TVD is the distance between ideal and noisy output distributions (0 is perfect). Exact density-matrix simulation with FakeTorino errors (Heron) or a depolarising model (ion).



Calibrated beats Qiskit L3 on **39/49** Heron circuits.

Takeaway. Counting 2q gates with weight 10 is almost as good as full calibration, a useful proxy when no calibration exists.

Paper Table B2

Result 3. Large circuits, 14–100 qubits

Relative to Qiskit L3

	Heron (22)			**ion** (25)	
	2q	all	ESP*	2q	all
Qiskit L1	×1.22	×1.13	×0.24	×1.40	×1.13
TKET	×1.07	×1.96	×0.27	×1.39	×1.87
ours from L1	×1.05	**×0.78**	×0.43	**×0.96**	**×0.68**
ours from L3	**×0.98**	**×0.81**	**×1.12**	×1.00	×1.00

45 Nam/VOQC circuits, 88 of 90 runs done. pf circuits excluded (Qiskit L3 drops their 10^{-7} rad rotations). *Over the 13 Heron circuits whose ESP is at least 10^{-3}.

Paper Table 6, `results/general/large_v1.jsonl`, `large_bqskit_v1.jsonl`

Result 3. Large circuits, 14–100 qubits

Relative to Qiskit L3

	Heron (22)			ion (25)	
	2q	all	ESP*	2q	all
Qiskit L1	×1.22	×1.13	×0.24	×1.40	×1.13
TKET	×1.07	×1.96	×0.27	×1.39	×1.87
ours from L1	×1.05	**×0.78**	×0.43	**×0.96**	**×0.68**
ours from L3	**×0.98**	**×0.81**	**×1.12**	×1.00	×1.00

45 Nam/VOQC circuits, 88 of 90 runs done. pf circuits excluded (Qiskit L3 drops their 10^{-7} rad rotations). *Over the 13 Heron circuits whose ESP is at least 10^{-3}.

BQSKit on the 14 circuits it finished (≤36 qubits)

Heron, vs. Qiskit L3	2q	all
BQSKit L3 (approximate)	**×0.91**	**×1.35**
ours from L3 (exact)	×0.98	**×0.82**

- **On L3, Heron.** 19 % fewer gates, fewer CZ on 20/22, never more.
- **From L1.** Fewer gates but lower ESP. At this scale Qiskit's L3 layout matters more than what we remove.

Takeaway. Larger circuits leave more to remove, and the guarantees hold unchanged.

Paper Table 6, `results/general/large_v1.jsonl`, `large_bqskit_v1.jsonl`

What it costs: run time per circuit

Median seconds per circuit

	60-circuit suite		large suite	
pipeline	Heron	ion	Heron	ion
Qiskit level 1	–	–	0.11	0.04
Qiskit level 3	0.03	0.01	0.12	0.05
TKET	0.30	0.86	8.2	9.6
BQSKit level 2 (8 workers)	14.7	4.8	132	34.2
BQSKit level 3 (8 workers)	53.6	11.2	842	138
ours, exact, from L1 (1 core)	10.4	3.0	853	187
ours, exact, from L3 (1 core)	6.0	4.8	117	57.1

The slowest single circuit took 6.4 h (100 qubits, single-threaded Python). This is the main cost at scale.

Paper Table B1. BQSKit's large-suite medians are over its 14 smaller circuits only.

Result 4. Robust over seeds, and what each part contributes

5 transpiler seeds, Heron (mean ± sd vs. L3)

ours on L3, all gates	$\times 0.868 \pm 0.005$
ours on L3, 2q gates	$\times 0.975 \pm 0.004$
ours on L3, ESP	$\times 1.03$ (1.03–1.04)
ours on L1, ESP	$\times \mathbf{1.10 \pm 0.11}$

Gate counts are stable. ESP from L1 follows Qiskit's L1 layout, and seed 0 ($\times 1.21$) was the best case.

Paper Table 5, `results/general/robust_{seeds,ablation}_v1.jsonl`

Result 4. Robust over seeds, and what each part contributes

5 transpiler seeds, Heron (mean ± sd vs. L3)

ours on L3, all gates	$\times 0.868 \pm 0.005$
ours on L3, 2q gates	$\times 0.975 \pm 0.004$
ours on L3, ESP	$\times 1.03$ (1.03–1.04)
ours on L1, ESP	$\times \mathbf{1.10 \pm 0.11}$

Gate counts are stable. ESP from L1 follows Qiskit's L1 layout, and seed 0 ($\times 1.21$) was the best case.

Ablation, gates removed on top of L3 (Heron)

full optimiser	**13.0 %**
contiguous windows	11.7 %
no resynthesis	8.2 %
window $w = 8$	7.9 %
database radius ≤ 3	6.2 %
no database	5.8 %
window $w = 4$	**1.3 %**

Takeaway. Window length and database depth do the work, the same two knobs as in Part II.

Paper Table 5, `results/general/robust_{seeds,ablation}_v1.jsonl`

Result 5. Against VOQC and QUESO, on their own benchmark

Nam gate set $\{H, X, R_z, \mathrm{CNOT}\}$, 26 circuits

tool	gates	change	CNOT
input	10,481		4,612
TKET	11,762	$+12.2\,\%$	4,489
Qiskit L3	10,391	$-0.9\,\%$	4,511
QUESO (5 min)	9,765	$-6.8\,\%$	4,465
ours	9,480	$-9.6\,\%$	4,426
VOQC	8,417	$-19.7\,\%$	4,314
VOQC, then ours	**7,833**	$-25.3\,\%$	**4,007**

Same input files as the published comparisons. All 159 outputs equal up to phase.

Paper Table 7, `results/logical/`

Result 5. Against VOQC and QUESO, on their own benchmark

Nam gate set $\{H, X, R_z, \mathrm{CNOT}\}$, 26 circuits

tool	gates	change	CNOT
input	10,481		4,612
TKET	11,762	$+12.2\,\%$	4,489
Qiskit L3	10,391	$-0.9\,\%$	4,511
QUESO (5 min)	9,765	$-6.8\,\%$	4,465
ours	9,480	$-9.6\,\%$	4,426
VOQC	8,417	$-19.7\,\%$	4,314
VOQC, then ours	**7,833**	$-25.3\,\%$	**4,007**

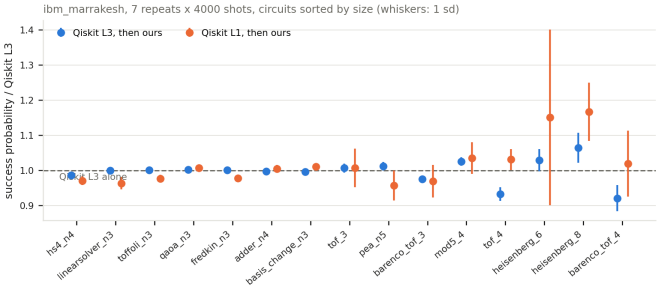
Same input files as the published comparisons. All 159 outputs equal up to phase.

- **VOQC wins alone.** It merges R_z gates across the whole circuit, our windows see 3 qubits.
- **We find what it misses.** mod5_4: 35 gates vs. 56.
- **Together** best overall, ours improves VOQC on 21/26.

Takeaway. Same lesson as with BQSKit. A global pass plus exact local cleanup beats either alone.

Paper Table 7, `results/logical/`

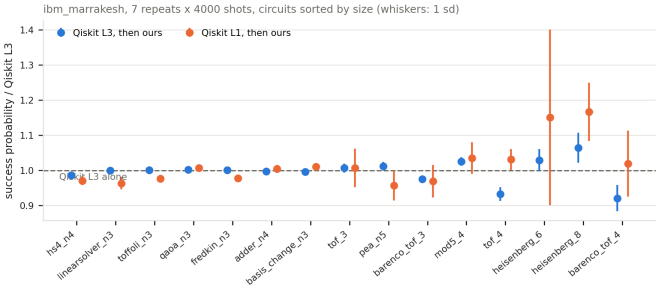
Result 6. First runs on a real IBM device



Probability of the correct output, relative to Qiskit L3 alone. Dots are means over 7 repeats.

`experiments/gen_hardware.py`, `results/hardware/ibm_marrakesh_v1/analysis.json`

Result 6. First runs on a real IBM device



Probability of the correct output, relative to Qiskit L3 alone. Dots are means over 7 repeats.

vs. L3 alone	mean	better
L3, then ours	$\times 0.996$	8/15
L1, then ours	$\times 1.015$	9/15
noise model	$\times 1.005$	

- 15 circuits, 3–8 qubits, IBM Heron r2.
- The effect is **within the noise**, as the noise model predicted.
- On small circuits we mostly remove 1q gates, which are nearly free.

`experiments/gen_hardware.py`, `results/hardware/ibm_marrakesh_v1/analysis.json`

Every output is verified, even beyond 25 qubits

outputs	how checked	count	result
paper benchmarks	statevector	250	**250/250**, max error $8 \cdot 10^{-11}$
60-circuit suite	statevector	720	**720/720**
Clifford+T	exact arithmetic	all	**exactly equal**
large suite, $\leq$24 qubits	statevector	57	**57/57**, max $1.4 \cdot 10^{-10}$
large suite, all	replay certificate	176	**171/176**, 5 pending
logical level, 6 tools	statevector / QCEC	159	**159/159** up to phase

Replay certificate. The optimiser logs every step. An independent checker replays the log with small matrices on a few wires (up to 544,922 steps, at most 6.8 s). Its cost grows with the steps, not the width.

`qcr/certify.py`, `test/test_certify.py`. The 5 pending were rejected by an older checker limit and not yet re-checked.

Part V

Wrap-up

Where it stands and what comes next.

Limitations

- Real hardware so far shows no measurable gain (15 small circuits, Result 6).
- Windows of at most 3 qubits, single-threaded Python, hours at 100 qubits.
- VOQC beats us alone on the Nam set, BQSKit on 2q gates.
- Quartz not run, QUESO at 5 min, BQSKit on the smaller large circuits only.

Limitations and next steps

Limitations

- Real hardware so far shows no measurable gain (15 small circuits, Result 6).
- Windows of at most 3 qubits, single-threaded Python, hours at 100 qubits.
- VOQC beats us alone on the Nam set, BQSKit on 2q gates.
- Quartz not run, QUESO at 5 min, BQSKit on the smaller large circuits only.

Next

- Hardware runs with more repeats, on circuits where we remove 2q gates.
- Reach VOQC's rotation merging and BQSKit's 2q counts exactly (phase-polynomial passes, larger windows).
- Quartz baseline, QUESO with its full hour.

Summary

1. The paper's benchmark gates are Clifford, so the optimum is computable. The method is **3–4.5**$\times$ off it, and the database radius is why.
2. A general optimiser with deep databases and convex windows reaches the optimum on **35/40** circuits without knowing the group.
3. On real algorithms it is a safe post-pass. After Qiskit L3 it never adds a 2q gate and removes 13–19 % of gates, and after BQSKit it removes 16 % more.
4. The gains are stable over seeds, and window length and database depth drive them.
5. Against VOQC and QUESO, ours is second alone and VOQC followed by ours is best.
6. Every output is verified. On real hardware the effect is not yet measurable.